Backend ObjectifSMART-AI

Le code explique **ligne par ligne** (version exhaustive)

Comprendre et faire evoluer le backend soi-meme
FastAPI - SQLAlchemy ORM - Architecture en couches

BMCI - Ecole Centrale Casablanca

0. Comment lire ce cours

Ce document explique **l'integralite** du backend ObjectifSMART-AI, **fichier par fichier**, avec le code reel et des explications detaillees. Objectif : que tu puisses lire, comprendre et **faire evoluer** le code seul.

- **Bloc gris** = code reel du projet.
- **Liste a puces** sous un bloc = explication des lignes/elements de ce bloc.
- **Encadre vert** = point cle a retenir.
- On commence par les concepts, puis on descend couche par couche : core, database, models, repositories, services, controllers, auth, middlewares, schemas, utils, main, tests.

1. Concepts fondamentaux

1.1 Backend, requete, reponse

Le **backend** est le programme serveur. Le navigateur (frontend Vue.js) envoie une **requete HTTP** (une methode + une URL, ex. GET /api/objectives, parfois avec un corps JSON). Le backend execute du code et renvoie une **reponse** (JSON, fichier, code d'erreur). Ici le backend est la **seule source de verite** : toute la securite est cote serveur.

1.2 FastAPI : routes et JSON

```
from fastapi import FastAPI

app = FastAPI()

@app.get("/hello")
def hello():
    return {"message": "bonjour"}
```

- **@app.get("/hello")** : decorateur = "quand GET /hello arrive, appelle cette fonction".
- On renvoie un **dict** ; FastAPI le transforme en **JSON** automatiquement.
- **APIRouter** : meme idee que app mais pour grouper des routes (un par domaine).

1.3 Depends : l'injection de dependances

FastAPI peut fournir une valeur a une route via Depends(fn) : il appelle fn AVANT la route. On l'utilise pour l'auth : si le token est invalide, fn leve une erreur 401 et la route n'est jamais executee.

1.4 Pydantic : valider les entrees

```
class LoginRequest(BaseModel):
    email: str
    password: str
```

Si le corps recu ne contient pas email, FastAPI repond 422 AVANT ta fonction. Tu n'ecris aucune validation a la main.

1.5 SQLAlchemy ORM

Un **ORM** fait correspondre une **table** a une **classe** et une **ligne** a un **objet**. On manipule des objets Python au lieu d'ecrire du SQL.

```
# objet Python au lieu de SQL :
db.scalars(select(User).where(User.email == email)).first()
```

- **Modele** (classe) = table. **Session** (db) = conversation avec la base.
- **select().where()** = requete + filtre. **commit** = valider, **rollback** = annuler.
- On evite les **injections SQL** et le code est plus lisible.

1.6 Architecture en couches

```
controllers  ->  routes HTTP (URL). PAS de logique.
services     ->  logique metier (les regles).
repositories ->  TOUTES les requetes base de donnees.
models       ->  description des tables (ORM).
database     ->  connexion / sessions.

transverses  : core, schemas, auth, middlewares, utils.
```

Regle d'or : controller -> service -> repository -> model. Jamais de SQL dans un controller, jamais de logique metier dans un repository.

2. Le voyage d'une requete

Deux cas selon que la requete a un CORPS ou non. Une requete **GET** n'a pas de corps -> rien a valider. Une requete **POST/PUT** a un corps JSON -> il est valide par un **schema** Pydantic AVANT d'entrer dans le controller.

Cas A - GET (sans corps) : GET /api/objectives

```
Navigateur  (GET /api/objectives, header Authorization: Bearer TOKEN)
   |
   v   [middleware taille]  ->  [middleware CORS]
controllers/objective_controller.py     (la route)
   |   (pas de corps  ->  aucun schema a valider)
   |   Depends(get_current_user)  ->  qui es-tu ?
   v
services/objective_service.py            (quel perimetre ? RBAC)
   |
   v
repositories/objective_repository.py     (requete SQLAlchemy)
   |
   v   models/objective.py  ->  base SQLite
   ^___ la reponse JSON remonte jusqu'au navigateur
```

Cas B - POST (avec corps) : POST /api/auth/login

```
Navigateur  (POST /api/auth/login, corps JSON {email, password})
   |
   v   [middleware taille]  ->  [middleware CORS]
schemas/auth_schema.py (LoginRequest)
   |   Pydantic valide le corps  ->  si invalide : 422 (la route n'est PAS appelee)
   v
controllers/auth_controller.py           (la route)
   |
   v
services/auth_service.py  ->  repositories  ->  models  ->  base SQLite
   ^___ la reponse JSON (token + profil) remonte au navigateur
```

- **Le schema n'est pas une etape controller->service->repository** : c'est une couche TRANSVERSE. Il valide l'ENTREE (le corps) et peut typer la SORTIE.
- C'est FastAPI qui l'execute automatiquement quand tu ecris def login(request: LoginRequest) : la validation a lieu AVANT le controller.
- Un corps invalide (champ manquant, mauvais type) -> reponse **422** sans que ton code ne s'execute. Tu n'ecris aucune validation a la main.

3. Couche core

3.1 core/config.py

```
import os

def load_dotenv(path: str = ".env") -> None:
    if not os.path.exists(path):
        return
    with open(path, "r", encoding="utf-8") as f:
        for line in f:
            line = line.strip()
            if not line or line.startswith("#") or "=" not in line:
                continue
            key, _, value = line.partition("=")
            key = key.strip()
            value = value.strip().strip('"').strip("'")
            if key and key not in os.environ:
                os.environ[key] = value
```

- Lit .env ligne par ligne ; ignore lignes vides et commentaires (#).
- partition("=") coupe "CLE=VALEUR" ; on enleve les guillemets de la valeur.
- if key not in os.environ : on n'ecrase jamais une variable systeme existante.

```
class Settings:
    def __init__(self):
        self.DB_PATH = os.getenv("DATABASE_PATH", "objectifsmart.db")
        self.TOKEN_EXPIRY_HOURS = int(os.getenv("SESSION_DURATION_HOURS", "24"))
        self.PBKDF2_ITERATIONS = int(os.getenv("PBKDF2_ITERATIONS", "200000"))
        self.SECRET_KEY = os.getenv("SECRET_KEY", "")
        _raw = os.getenv("ALLOWED_ORIGINS", "").strip()
        if not _raw or _raw == "*":
            raise RuntimeError("ALLOWED_ORIGINS doit etre defini...")
        self.ALLOWED_ORIGINS = [o.strip() for o in _raw.split(",") if o.strip()]
        self.MAX_REQUEST_SIZE_BYTES = int(os.getenv("MAX_REQUEST_SIZE_MB","20"))*1024*1024

load_dotenv()
settings = Settings()
```

- os.getenv("X", defaut) lit une variable d'env (toujours du texte) ; int(...) la convertit en nombre.
- Le bloc if not _raw or == "*" **refuse de demarrer** si CORS est trop permissif (un "*" + cookies = faille CSRF).
- load_dotenv() puis settings = Settings() : execute une seule fois. Partout ailleurs : from app.core.config import settings.

3.2 core/security.py

```
def hash_password(password: str) -> str:
    salt = secrets.token_bytes(16)
    dk = hashlib.pbkdf2_hmac("sha256", password.encode("utf-8"),
                             salt, settings.PBKDF2_ITERATIONS)
    return f"pbkdf2_sha256${settings.PBKDF2_ITERATIONS}${salt.hex()}${dk.hex()}"
```

- On ne stocke jamais le mot de passe en clair, seulement son empreinte (hash).
- **salt** : 16 octets aleatoires uniques -> deux memes mots de passe donnent deux hash differents.
- **pbkdf2_hmac** : applique SHA-256 des milliers de fois (lent volontairement) -> brute-force tres couteux.
- Le format encode tout : algo$iterations$sel$hash pour pouvoir re-verifier.

```
def verify_password(password, stored_hash) -> bool:
    if not stored_hash:
        return False
    if _is_legacy_sha256(stored_hash):
        legacy = hashlib.sha256(password.encode()).hexdigest()
        return hmac.compare_digest(legacy, stored_hash)
    algo, iterations, salt_hex, hash_hex = stored_hash.split("$")
    dk = hashlib.pbkdf2_hmac("sha256", password.encode(),
                             bytes.fromhex(salt_hex), int(iterations))
    return hmac.compare_digest(dk.hex(), hash_hex)
```

- On recalcule le hash du mot de passe fourni avec le MEME sel, puis on compare.
- **hmac.compare_digest** : comparaison a temps constant (empeche les attaques temporelles).
- **_is_legacy_sha256** : gere l'ancien format (64 hex) -> compatibilite et migration.

```
        def needs_rehash(stored_hash) -> bool:
            return _is_legacy_sha256(stored_hash)

        def generate_token() -> str:
            return secrets.token_urlsafe(32)

        _failed_login_attempts: Dict[str, List[float]] = {}

        def is_login_allowed(identifier) -> bool:
            now = time.time()
            window = now - settings.LOGIN_LOCKOUT_MINUTES * 60
            attempts = [t for t in _failed_login_attempts.get(identifier, []) if t > window]
            _failed_login_attempts[identifier] = attempts
            return len(attempts) < settings.MAX_LOGIN_ATTEMPTS
```

- **needs_rehash** : indique si un hash doit etre migre vers PBKDF2 (fait a la connexion).
- **generate_token** : jeton de session aleatoire et sur.
- **Anti brute-force** : un dict garde les horodatages d'echecs par identifiant ; on ne compte que ceux dans la fenetre recente ; au-dela de N echecs -> connexion bloquee (429).
- record_failed_login et reset_login_attempts completent ce mecanisme.

4. Couche database

4.1 database/base.py

```
from sqlalchemy.orm import DeclarativeBase

class Base(DeclarativeBase):
    pass
```

Classe mere de tous les modeles. SQLAlchemy tient le registre des tables dans Base.metadata. Isolee pour eviter les imports circulaires.

4.2 database/session.py

```
DATABASE_URL = f"sqlite:///{settings.DB_PATH}"
engine = create_engine(DATABASE_URL,
                       connect_args={"check_same_thread": False}, future=True)
SessionLocal = sessionmaker(bind=engine, autoflush=False, autocommit=False,
                            expire_on_commit=False, class_=Session, future=True)
```

- **engine** : sait se connecter au fichier SQLite.
- **check_same_thread=False** : autorise l'usage depuis plusieurs threads (serveur web).
- **SessionLocal** : fabrique de sessions. **expire_on_commit=False** : on peut relire un objet apres commit (pratique pour renvoyer un dict).

```
def get_db():
    db = SessionLocal()
    try:
        yield db
    finally:
        db.close()

@contextmanager
def db_session():
    db = SessionLocal()
    try:
        yield db
        db.commit()
    except Exception:
        db.rollback()
        raise
    finally:
        db.close()
```

- **get_db** : dependance FastAPI (une session par requete, fermee a la fin).
- **db_session** : pour les services. with db_session() as db: -> commit si OK, rollback si exception, close toujours. C'est ici que les transactions sont gerees.

4.3 database/init_db.py

```
from app.database.base import Base
from app.database.session import engine
import app.models   # enregistre toutes les tables

def init_db() -> None:
    Base.metadata.create_all(bind=engine)
```

- import app.models : importe tous les modeles -> ils s'enregistrent sur Base.metadata.
- **create_all** : cree les tables manquantes. Idempotent -> ne touche pas l'existant.
- Appele au demarrage (main.py) et par les tests.

5. Couche models (les 7 tables)

Chaque modele = une table. Schema reel : identifiants en **texte** (ex. user-ab12), dates en **texte** (ISO), analyses stockees en **JSON serialise**.

5.1 models/user.py

```
class User(Base):
    __tablename__ = "users"
    id: Mapped[str] = mapped_column(String, primary_key=True, index=True)
    email: Mapped[str] = mapped_column(String, unique=True, index=True, nullable=False)
    password_hash: Mapped[str] = mapped_column(String, nullable=False)
    name: Mapped[str] = mapped_column(String, nullable=False)
    role: Mapped[str] = mapped_column(String, nullable=False)
    manager_id: Mapped[str | None] = mapped_column(String, ForeignKey("users.id"))
    team: Mapped[str | None] = mapped_column(String, nullable=True)
    entity: Mapped[str | None] = mapped_column(String, nullable=True)
    created_at: Mapped[str | None] = mapped_column(
        String, server_default=text("CURRENT_TIMESTAMP"))
    __table_args__ = (CheckConstraint("role IN ('COLLAB','MANAGER','HR')"),)
    manager = relationship("User", remote_side="User.id", back_populates="subordinates")
    subordinates = relationship("User", back_populates="manager")
    objectives = relationship(back_populates="user", cascade="all, delete-orphan")
    sessions = relationship(back_populates="user", cascade="all, delete-orphan")
```

- **__tablename__** : nom de la table. **Mapped[..] + mapped_column(..)** : une colonne.
- **primary_key** : id unique (texte ici). **unique** : pas deux emails identiques.
- **index** : recherche rapide. **nullable=False** : obligatoire. **str | None** : optionnel.
- **ForeignKey("users.id")** : manager_id pointe vers un autre User (auto-reference).
- **server_default CURRENT_TIMESTAMP** : date mise par la base si non fournie.
- **CheckConstraint** : la base refuse tout role hors liste.
- **relationship** : lien pratique (pas une colonne). user.objectives = liste des objectifs. **cascade delete-orphan** : supprimer un User supprime ses objectifs/sessions.
- Helper to_dict() : renvoie un dict SANS le password_hash.

5.2 models/session.py

```
class Session(Base):
    __tablename__ = "sessions"
    token: Mapped[str] = mapped_column(String, primary_key=True)
    user_id: Mapped[str] = mapped_column(
        String, ForeignKey("users.id", ondelete="CASCADE"), nullable=False)
    expires_at: Mapped[str] = mapped_column(String, nullable=False)
    created_at: Mapped[str | None] = mapped_column(
        String, server_default=text("CURRENT_TIMESTAMP"))
    user = relationship(back_populates="sessions")
```

- Le **token** EST la cle primaire. **ondelete=CASCADE** : supprimer un user supprime ses sessions.
- **expires_at** : date d'expiration ; comparee a 'maintenant' a chaque requete protegee.

5.3 models/objective.py

```
class Objective(Base):
    __tablename__ = "objectives"
    id: Mapped[str] = mapped_column(String, primary_key=True)
    user_id: Mapped[str] = mapped_column(String, ForeignKey("users.id", ondelete="CASCADE"))
    title: Mapped[str] = mapped_column(Text, nullable=False)
    description / context : Mapped[str | None] = mapped_column(Text)
    status = mapped_column(String, server_default=text("'draft'"))
    progress = mapped_column(String, server_default=text("'not_started'"))
    smart_analysis / alignment_analysis : Text   # JSON serialise
    analyzed_by_role / analyzed_by_user_id : String
    data_scope = mapped_column(String, server_default=text("'COLLAB'"))
    entity / manager_name / period / import_batch_id : String
    created_at / updated_at : CURRENT_TIMESTAMP
    user = relationship(back_populates="objectives")
```

- **Text** (vs String) : pour les longs textes (titre, description, JSON d'analyse).
- **smart_analysis / alignment_analysis** : le resultat IA, stocke en JSON texte ; ce sont les services qui le serialisent/deserialisent.
- **data_scope** + **import_batch_id** : metadonnees d'ISOLATION. Les imports RH sont marques 'HR' ; la vue Manager les exclut. C'est ce qui cloisonne RH et Manager.

5.4 models/company_objective.py

```
class CompanyObjective(Base):
    __tablename__ = "company_objectives"
    id (PK) / title / description / department
    priority: Integer (default 1) ; weight: Float (default 1.0)
    created_by: ForeignKey("users.id") ; import_batch_id ; created_at
```

Catalogue des objectifs strategiques de l'entreprise. Sert de reference a l'analyse d'alignement (on compare chaque objectif individuel a ce catalogue).

5.5 models/analysis_result.py (3 tables)

Trois tables SEPAREES pour garantir l'isolation (un manager ne voit jamais les analyses RH). Un **mixin** factorise les colonnes communes.

```
class AnalysisResultMixin:
    id (PK) ; objective_id ; objective_title
    smart_analysis ; alignment_analysis            # JSON texte
    smart_score ; alignment_score : Float
    s_score ; m_score ; a_score ; r_score ; t_score : Integer
    issues_count ; top_issue ; analyzed_at

class HRAnalysisResult(AnalysisResultMixin, Base):
    __tablename__ = "hr_analysis_results"
class ManagerAnalysisResult(AnalysisResultMixin, Base):
    __tablename__ = "manager_analysis_results"
    manager_id: Mapped[str]
class CollabAnalysisResult(AnalysisResultMixin, Base):
    __tablename__ = "collab_analysis_results"
    user_id ; objective_description ; rewrite_fr ; rewrite_en
```

- **Mixin** : classe "morceau" ; ses colonnes sont ajoutees aux 3 tables sans copier-coller.
- Chaque table ajoute sa specificite : manager_id, user_id...
- AnalysisResult = HRAnalysisResult : un alias generique.

5.6 models/audit_log.py

```
class AuditEventType(str, Enum):
    LOGIN_SUCCESS = "LOGIN_SUCCESS" ; LOGIN_FAILED ; LOGOUT_MANUAL
    SESSION_EXPIRED ; ACCESS_DENIED ; ROLE_ACCESS_DENIED ; ...

class AuditLog(Base):
    __tablename__ = "audit_logs"
    id: Mapped[int] = mapped_column(Integer, primary_key=True, autoincrement=True)
    user_id ; username ; role ; event_type ; event_status
    ip_address ; user_agent ; endpoint ; http_method
    description ; session_ref ; created_at
```

- **Enum** : liste fermee des types d'evenements traçables (evite les fautes de frappe).
- **id Integer autoincrement** : seule table a id numerique (journal sequentiel).
- **session_ref** : hash court du token (jamais le token brut). **description** : filtree.

5.7 models/hr_import.py

```
class HRImport(Base):
    __tablename__ = "hr_imports"
    id (PK) ; import_type ("objectives"|"strategy")
    filename ; records_count ; user_id (FK) ; created_at
```

Journalise chaque import Excel RH (alimente l'historique /api/hr/imports).

6. Couche repositories (toutes les requetes)

Chaque fonction reçoit db (la session, ouverte par le service) et ne committe jamais. C'est le SEUL endroit ou l'on parle a la base.

6.1 user_repository.py

```
        def get_by_email(db, email):
            return db.scalars(select(User).where(User.email == email)).first()

        def get_by_valid_session(db, token, now_iso):
            stmt = (select(User)
                    .join(SessionModel, User.id == SessionModel.user_id)
                    .where(SessionModel.token == token,
                           SessionModel.expires_at > now_iso))
            return db.scalars(stmt).first()

        def create(db, *, user_id, email, password_hash, name, role,
                   manager_id=None, entity=None):
            user = User(id=user_id, email=email, password_hash=password_hash,
                        name=name, role=role, manager_id=manager_id, entity=entity)
            db.add(user); db.flush(); return user
```

- **get_by_email** : select(User).where(...) construit la requete ; db.scalars(...).first() l'execute et renvoie le 1er User (ou None).
- **get_by_valid_session** : jointure users/sessions ; verifie token present ET non expire. C'est le coeur de l'authentification.
- **create** : db.add marque pour insertion ; db.flush() envoie l'ordre maintenant (detecte un email en double tout de suite) sans committer.
- Le * force a NOMMER les arguments -> appels lisibles, pas d'erreur d'ordre.

Autres fonctions : get_by_id, update_password_hash, is_manager, get_manager_id, list_managers, get_team_ids, get_all_ids, get_team_members, get_all_members - toutes sur le meme modele (un select + filtre).

6.2 session_repository.py

```
        def create(db, token, user_id, expires_at):
            db.add(SessionModel(token=token, user_id=user_id, expires_at=expires_at))

        def delete_by_user(db, user_id):
            db.execute(delete(SessionModel).where(SessionModel.user_id == user_id))
```

- **create** : ajoute une session. **delete_by_user** : supprime toutes les sessions d'un user (utilise au login pour garantir une session unique, et au logout).

6.3 objective_repository.py

```
        def list_by_user_ids(db, user_ids):
            stmt = (select(Objective).options(selectinload(Objective.user))
                    .where(Objective.user_id.in_(user_ids))
                    .order_by(Objective.created_at.desc()))
            return list(db.scalars(stmt).all())

        def list_team_active(db, team_ids):   # vue MANAGER : exclut les imports RH
            stmt = (select(Objective).options(selectinload(Objective.user))
                    .where(Objective.user_id.in_(team_ids),
                           or_(Objective.data_scope.is_(None), Objective.data_scope != "HR"),
                           or_(Objective.import_batch_id.is_(None),
                               Objective.import_batch_id.not_like("obj-%")))
                    .order_by(Objective.created_at.desc()))
            return list(db.scalars(stmt).all())
```

- **.in_(user_ids)** : equivaut a SQL "WHERE user_id IN (...)".
- **selectinload(Objective.user)** : charge l'utilisateur lie en meme temps (evite des requetes supplementaires quand on lira obj.user.name).

- **list_team_active** : applique l'ISOLATION (exclut data_scope='HR' et les lots 'obj-%').
- **merge(db, values)** = upsert (INSERT OR REPLACE) par cle primaire. **update_fields**, **delete_by_id**, et des agregations (count_by_user_ids, get_progress_counts...) completent ce repository.

6.4 company_objective_repository.py & hr_import_repository.py

- **company_objective_repository** : list_all, list_for_alignment (projection {id,title,description} pour l'IA), create, merge.
- **hr_import_repository** : create (journalise un import), list_recent (historique).

6.5 analysis_repository.py - ecritures ORM + reporting text()

```
        def insert_hr(db, values):  db.add(HRAnalysisResult(**values))
        def upsert_hr(db, values):  return db.merge(HRAnalysisResult(**values))
        #  idem insert_manager / upsert_manager / insert_collab
```

Pour les ecritures, on construit le modele a partir d'un dict (**values). **insert** = ajout ; **upsert** = ajout-ou-remplacement (merge par cle).

```
        def manager_stats(db, manager_id):
            sql = text("""
                SELECT COUNT(*) AS total_analyzed, AVG(smart_score) AS avg_smart_score
                FROM manager_analysis_results WHERE manager_id = :manager_id
            """)
            row = db.execute(sql, {"manager_id": manager_id}).mappings().first()
            return dict(row) if row else {}
```

- Reporting complexe (jointures, AVG, regroupement par mois) -> on utilise **text()** : du SQL ecrit a la main MAIS execute par la session, et TOUJOURS dans le repository.
- **:manager_id** : parametre nomme. On ne colle jamais une valeur dans la chaine -> protection contre l'injection SQL.
- **.mappings().first()** : 1ere ligne sous forme de dict (colonne -> valeur).
- Helper prive _hr_filters(...) : construit la clause WHERE du perimetre RH (data_scope='HR' OU import_batch_id LIKE 'obj-%') + filtres optionnels, avec parametres nommes.
- Les fonctions hr_analytics, hr_objectives_details, hr_export_*, list_manager_results, etc. suivent ce schema.

6.6 audit_repository.py

```
        def insert(db, values):
            db.add(AuditLog(**values))

        def fetch(db, *, limit=100, offset=0, event_type=None, user_id=None):
            stmt = select(AuditLog)
            if event_type: stmt = stmt.where(AuditLog.event_type == event_type)
            if user_id:    stmt = stmt.where(AuditLog.user_id == user_id)
            stmt = stmt.order_by(AuditLog.created_at.desc()).limit(min(limit,500)).offset(offset)
            return list(db.scalars(stmt).all())
```

- **fetch** : requete construite par etapes (on ajoute les .where seulement si un filtre est fourni). **limit/offset** : pagination ; on plafonne a 500.

7. Couche services (la logique metier)

7.1 auth_service.py

```
        def resolve_session_user(token):
            with db_session() as db:
                user = user_repository.get_by_valid_session(db, token, datetime.now().isoformat())
                return user.to_dict() if user else None
```

Verifie un token et renvoie l'utilisateur (dict) ou None. Utilise par la dependance d'auth.

```
        def login(request, meta):
            identifier = (request.email or "").strip().lower()
            if not security.is_login_allowed(identifier):
                audit.log_event(LOGIN_FAILED, ...); raise HTTPException(429, "Trop de tentatives...")
            provider = get_identity_provider()
            profile = provider.authenticate({"email": request.email, "password": request.password})
            if not profile:
                security.record_failed_login(identifier)
                audit.log_event(LOGIN_FAILED, ...); raise HTTPException(401, "...incorrect.")
            security.reset_login_attempts(identifier)
            user_id = profile["external_id"]; token = security.generate_token()
            expires = datetime.now() + timedelta(hours=settings.TOKEN_EXPIRY_HOURS)
            with db_session() as db:
                user = user_repository.get_by_id(db, user_id)
                if user and security.needs_rehash(user.password_hash):
                    user_repository.update_password_hash(db, user_id,
                                                         security.hash_password(request.password))
                session_repository.delete_by_user(db, user_id)
                session_repository.create(db, token, user_id, expires.isoformat())
                user_public = _public_user(user); name = user.name
            audit.log_event(LOGIN_SUCCESS, f"Connexion de {name}", ...)
            return {"success": True, "token": token, "user": user_public}
```

- 1) Rate limiting : trop d'echecs -> 429 sans verifier le mot de passe.
- 2) **get_identity_provider().authenticate** : delegue la verification (local/LDAP).
- 3) Echec -> enregistre la tentative, journalise, 401. Succes -> remet le compteur a zero.
- 4) Dans db_session() : migration de hash si besoin, suppression des anciennes sessions (session unique), creation de la nouvelle.
- 5) On renvoie le token + le profil public (jamais le hash). Tout le bloc DB = une transaction.

register : force le role COLLAB (anti-escalade), exige un manager existant, cree user + session, gere l'email en double (IntegrityError -> 400). **logout** : supprime les sessions + audit.

7.2 permission_service.py (RBAC)

```
        def get_accessible_user_ids(current_user):
            if current_user["role"] == "COLLAB":
                return [current_user["id"]]
            if current_user["role"] == "MANAGER":
                with db_session() as db:
                    return user_repository.get_team_ids(db, current_user["id"])
            if current_user["role"] == "HR":
                with db_session() as db:
                    return user_repository.get_all_ids(db)
            return []
```

- Definit le PERIMETRE de donnees : COLLAB=lui seul, MANAGER=son equipe, HR=tout le monde.
- require_role, require_admin, check_collab_access, check_manager_access levent une 403 si l'acces est interdit.

7.3 audit_service.py

```
        def log_event(event_type, description="", ..., token=None):
            try:
                values = {..., "description": _sanitize(description),
                          "session_ref": session_ref(token), ...}
                with db_session() as db:
                    audit_repository.insert(db, values)
            except Exception as e:
                print(f"[AUDIT][WARN] {e}")   # ne casse jamais la requete
```

- **_sanitize** : remplace une description contenant 'password'/'token'/... par '[DESCRIPTION FILTREE]'.
- **session_ref** : hash court du token (jamais le token brut).
- **try/except** : journaliser ne doit jamais faire echouer une requete metier.
- **fetch_logs** : relit via le repository et renvoie des dicts (route admin).

7.4 user_service.py

- **get_managers_public()** : liste des managers pour l'inscription, en n'exposant que id/name/team (jamais l'email -> anti-enumeration).
- **get_team_members(current_user)** : membres selon le role (MANAGER=son equipe, HR=tous).

7.5 objective_service.py

- **list_objectives** : calcule le perimetre (permission_service), lit via le repository, deserialise les analyses JSON, ajoute nom/email du proprietaire.
- **create / get / update / delete** : CRUD avec controle RBAC (proprietaire ou HR).
- **company-objectives** : liste, creation, import batch (merge).
- **compute_team_stats** : calcule total, taux SMART, alignement moyen, repartition d'avancement (reutilise par le domaine manager).
- Helper _parse_analyses : transforme le JSON texte en objets Python pour la reponse.

7.6 manager_service.py

- **get_objectives** : objectifs d'equipe (hors imports RH) via list_team_active.
- **get_team / get_team_stats** : membres et statistiques de l'equipe.
- **get_analysis_results / get_collab_analysis_results** : lit les tables privees + statistiques ; helper _finalize_result (parse JSON, construit un titre d'affichage).
- **analyze_personal** : analyse l'objectif perso du manager (IA) puis l'enregistre.
- **analyze_objective(id)** : verifie que l'objectif appartient a l'equipe, lance l'IA, met a jour l'objectif et enregistre le resultat (upsert).
- **export_data** : recupere les donnees puis delegue a export_service (Excel/PDF).
- Helpers _smart_flags_score (S+M+A+R+T)/5*100 et _smart_values (sous-scores 0/1, nombre d'issues, issue principale).

7.7 hr_service.py

- **analyze_personal / get_analysis_results** : analyses personnelles RH (table hr_analysis_results).
- **import_strategy** : lit l'Excel, mappe les colonnes, upsert dans company_objectives, journalise l'import.
- **import_objectives** : cree les comptes manquants avec un mot de passe ALEATOIRE FORT par compte (secrets.token_urlsafe), upsert les objectifs (data_scope='HR'), renvoie les credentials temporaires dans created_users.
- **build_batch_generator** : prepare l'analyse en lot et renvoie un GENERATEUR qui emet des lignes 'data: ...' (flux SSE) : start, progress, done. Chaque resultat est ecrit en base.
- **get_analytics / get_objectives_details** : tableaux de bord (delegue au repository text()).
- **export_excel / export_pdf** : recupere les donnees via le repository, delegue la generation a export_service.

7.8 ai_service.py (Ollama)

```
        def call_ollama(prompt, system):
            response = requests.post(settings.OLLAMA_URL, json={
                "model": settings.OLLAMA_MODEL, "system": system,
                "prompt": f"/no_think\n{prompt}", "stream": False, "format": "json"},
                timeout=180)
            response.raise_for_status()
            return extract_json_from_text(response.json().get("response", "{}"))
```

- **call_ollama** : appelle le modele local. Erreurs traduites : 503 (Ollama eteint), 504 (timeout), 500 (reponse invalide).
- **extract_json_from_text** : recupere le JSON meme si le modele ajoute du texte/markdown.
- **has_measure / has_time / enforce_consistency** : heuristiques qui CORRIGENT le modele (ex. critere M=true sans chiffre -> remis a false) et recalculent overall.
- **analyze_smart_internal** -> {S,M,A,R,T,overall,confidence,issues,rewrite_fr,rewrite_en}. **analyze_alignment_internal** -> {alignment_score, top_matches, gaps}.
- Fichier PUR : aucun acces base (il reçoit/retourne des donnees).

7.9 excel_import_service.py

- **ExcelColumnDetector** : note chaque colonne (mots-cles FR/EN, similarite, longueur des valeurs) pour deviner la colonne 'objectif' et 'description'.
- **scan_excel_file** : explore toutes les feuilles. **extract_objectives_from_excel** : extrait, nettoie, dedoublonne.
- **analyze_objectives_batch** : SMART + alignement pour chaque objectif (via ai_service).
- **run_import_analyze** : pipeline complet -> stocke les resultats dans la table privee du ROLE (manager/hr/collab) via _store_result_by_role.
- **run_import_export** : renvoie un Excel enrichi des colonnes d'analyse ; **run_scan** : previsualisation.

7.10 export_service.py

- PUR : reçoit des donnees deja extraites et fabrique des documents.
- **generate_manager_excel/pdf** : rapport equipe (3 feuilles Excel / tableaux PDF).
- **generate_hr_excel/pdf** : rapport RH (4 feuilles : objectifs, stats entite, stats manager, points bloquants).
- Renvoie une **StreamingResponse** avec Content-Disposition: attachment (telechargement). Verifie la presence d'openpyxl/reportlab (sinon 500).

8. Couche controllers (routes fines)

8.1 auth_controller.py

```
router = APIRouter(prefix="/api", tags=["Authentification"])

def get_current_user(request=None, authorization: Optional[str] = Header(None)):
    ip, ua, endpoint, method = request_meta(request)
    if not authorization:
        audit.log_event(ACCESS_DENIED, "Acces sans token", ...)
        raise HTTPException(401, "Token manquant...")
    token = authorization.replace("Bearer ", "")
    user = auth_service.resolve_session_user(token)
    if not user:
        audit.log_event(SESSION_EXPIRED, "...", token=token)
        raise HTTPException(401, "Session expiree ou invalide.")
    return user

@router.post("/auth/login")
def login(request: LoginRequest, http_request=None):
    return auth_service.login(request, request_meta(http_request))
```

- **APIRouter(prefix="/api")** : toutes les routes commencent par /api.
- **get_current_user** : pas une route, une DEPENDANCE reutilisee par tous les controllers.
- **Header(None)** : lit le header Authorization. Absent -> 401. On retire 'Bearer '.
- **resolve_session_user** : le service valide le token. Invalide -> 401.
- **login** : une ligne -> tout le travail est dans le service. C'est le patron de TOUTES les routes.

8.2 admin_controller.py

```
router = APIRouter(prefix="/api/admin", tags=["Administration"])

@router.get("/audit-logs")
def get_audit_logs(limit=100, offset=0, event_type=None, user_id=None,
                   current_user=Depends(get_current_user)):
    require_admin(current_user)
    logs = audit.fetch_logs(limit=limit, offset=offset,
                            event_type=event_type, user_id=user_id)
    return {"total": len(logs), "logs": logs}
```

- **require_admin** : refuse l'acces (403) si l'utilisateur n'est pas HR/ADMIN.
- Les parametres (limit, offset...) sont des **query params** (?limit=50) lus par FastAPI.

8.3 objective / manager / hr / ai controllers

- Tous suivent le meme patron : une route = Depends(get_current_user) + un appel au service correspondant.
- **hr_controller** et **ai_controller** lisent les fichiers uploades : content = await file.read() puis appellent le service.
- **hr_controller.analyze_batch** emballe le generateur du service dans une StreamingResponse(media_type="text/event-stream") (le SSE).

8.4 main.py - l'assemblage

```
@asynccontextmanager
async def lifespan(app):
    init_db()          # cree les tables manquantes au demarrage
    yield

def create_app():
    app = FastAPI(title=settings.APP_NAME, version="3.0.0", lifespan=lifespan)
    app.add_middleware(LimitRequestSizeMiddleware, max_bytes=settings.MAX_REQUEST_SIZE_BYTES)
    app.add_middleware(CORSMiddleware, allow_origins=settings.ALLOWED_ORIGINS, ...)
    for r in (auth_router, admin_router, objective_router,
              manager_router, hr_router, ai_router):
        app.include_router(r)
    @app.get("/health")
    def health(): return {"status": "ok", "app": settings.APP_NAME}
    return app

app = create_app()
```

- **lifespan** : code au demarrage -> init_db().
- **add_middleware** : limite de taille PUIS CORS (l'ordre compte).
- **include_router** : on monte chaque controller (on 'branche' les domaines).
- **app = create_app()** : cible de uvicorn app.main:app.

9. Couches transverses

9.1 auth/ - abstraction d'identite

```
class IdentityProvider(ABC):
    @abstractmethod
    def authenticate(self, credentials): ...
    @abstractmethod
    def get_user_groups(self, external_id): ...

def get_identity_provider():
    if os.getenv("AUTH_PROVIDER", "local") == "ldap":
        from app.auth.providers.ldap_provider import LDAPProvider
        return LDAPProvider(...)
    from app.auth.providers.local_provider import LocalDatabaseProvider
    return LocalDatabaseProvider()
```

- **ABC / @abstractmethod** : un CONTRAT. Toute implementation doit fournir authenticate.
- **LocalDatabaseProvider** : verifie email+mot de passe en base (via user_repository).
- **LDAPProvider** : squelette pour l'annuaire de la banque (a completer plus tard).
- **get_identity_provider** : la fabrique choisit l'implementation selon AUTH_PROVIDER. auth_service ne sait pas laquelle -> on pourra brancher LDAP sans toucher au reste.
- **map_groups_to_role** : convertira des groupes AD en roles applicatifs.

9.2 middlewares/request_size.py

```
class LimitRequestSizeMiddleware(BaseHTTPMiddleware):
    async def dispatch(self, request, call_next):
        cl = request.headers.get("content-length")
        if cl is not None and int(cl) > self.max_bytes:
            return JSONResponse(status_code=413, content={"detail": "Requete trop volumineuse..."})
        return await call_next(request)
```

- Un **middleware** s'execute pour TOUTES les requetes, avant les routes.
- On lit Content-Length AVANT de lire le corps -> defense anti-DoS rapide (413).

9.3 schemas/ et utils/

- **schemas/** : classes Pydantic (LoginRequest, RegisterRequest, ObjectiveCreate, ObjectiveUpdate, CompanyObjectiveCreate, SmartRequest, AlignmentRequest, ManagerPersonalAnalysisRequest, HRPersonalAnalysisRequest, UserPublic). Validation pure.
- **utils/helpers.py** : safe_int/safe_float (conversions sures), calculate_smart_total/is_smart_valid (score SMART), request_meta (extrait ip/user_agent/endpoint/method pour l'audit).

9.4 tests/ (app/tests/)

- **conftest.py** : prepare une base SQLite TEMPORAIRE (jamais la prod), schema cree via init_db() ; fixtures client, make_user, auth_token ; nettoyage des tables entre chaque test.
- **test_security** (hash/tokens/rate-limit/413), **test_audit** (filtrage anti-fuite), **test_permissions** (RBAC), **test_auth_endpoints** (401/login/register/429), **test_hr_import** (mots de passe aleatoires distincts). 31 tests, tous verts.

10. Tutoriel : ajoute un endpoint de A a Z

But : GET /api/objectives/count = nombre d'objectifs de l'utilisateur connecte. La demarche est TOUJOURS la meme, couche par couche.

Etape 1 - Repository (la requete)

```
# objective_repository.py
def count_for_user(db, user_id):
    stmt = select(func.count()).select_from(Objective).where(
        Objective.user_id == user_id)
    return int(db.scalar(stmt) or 0)
```

Etape 2 - Service (la regle)

```
# objective_service.py
def count_my_objectives(current_user):
    with db_session() as db:
        n = objective_repository.count_for_user(db, current_user["id"])
    return {"count": n}
```

Etape 3 - Controller (la route)

```
# objective_controller.py
@router.get("/objectives/count")
def count_objectives(current_user=Depends(get_current_user)):
    return objective_service.count_my_objectives(current_user)
```

Etape 4 - Verifier

- Lance : python -m uvicorn app.main:app --reload --port 5001.
- Ouvre http://localhost:5001/docs : ta route apparait automatiquement.
- Ajoute un test dans app/tests/ sur le modele des tests existants.

> **Methode universelle :** requete -> **repository** ; regle -> **service** ; URL -> **controller** ; validation d'entree -> **schema**. Jamais de SQL dans le controller, jamais de logique dans le repository.

11. Glossaire

- **Endpoint/route** : une URL geree par le backend.
- **JSON** : format texte d'echange (dictionnaires/listes).
- **ORM** : mappe tables et classes (SQLAlchemy).
- **Session (DB)** : conversation transactionnelle ; **commit/rollback** = valider/annuler.
- **RBAC** : controle d'acces par role (COLLAB/MANAGER/HR).
- **Depends** : valeur fournie automatiquement a une route.
- **Middleware** : code execute pour toutes les requetes, avant les routes.
- **Token Bearer** : jeton d'auth dans le header Authorization.
- **Upsert / merge** : inserer ou remplacer par cle primaire.
- **SSE** : flux d'evenements serveur (lignes 'data: ...').
- **Codes HTTP** : 200 OK, 401 non authentifie, 403 interdit, 404 introuvable, 413 trop volumineux, 422 entree invalide, 429 trop de tentatives, 500 erreur serveur.

Tu as desormais la carte complete du backend, fichier par fichier. Ouvre le code reel a cote de ce cours : tu reconnaitras chaque element. Bonne continuation !